

UD01.04: Literales, variables, expresiones y operadores

Contenido

1. Literales, variables y tipos de datos	2
1.1 Literales	2
1.2 Variables	2
1.3 Conversión de tipos.....	4
1.4 Identificadores válidos y convenciones	4
2. Operadores	6
2.1 Operadores aritméticos	6
2.2 Operadores relacionales (comparación)	6
2.3 Operadores lógicos	7
2.4 Operadores de asignación compuesta.....	7
2.5 Operadores de pertenencia (in, not in)	7
2.6 Operadores de identidad (is, is not)	8
3. Expresiones y precedencia	9
3.1 Concepto de expresión	9
3.2 Expresiones anidadas y uso de paréntesis	9
3.3 Precedencia y asociatividad de operadores.....	9
3.4 Borrado de variables y ámbito (scope)	10
4. Función eval() (Opcional)	12
4.1 ¿Qué es eval()?.....	12
4.2 Riesgos de seguridad.....	12

1. Literales, variables y tipos de datos

1.1 Literales

Los **literales** son valores constantes que se escriben directamente en el código. Son la forma más básica de representar datos.

- **Numéricos:**
 - Enteros (int): 42, -10
 - Flotantes (float): 3.14, 0.5, 1.2e-3
- **Cadenas de caracteres (str):** "Hola mundo", 'Python', """Texto multilinea"""
- **Booleanos (bool):** True, False
- **Especiales:** None (representa la ausencia de valor).

Ejemplos de literales

```
edad = 25      # Literal entero
nombre = "Ana"  # Literal cadena
activo = True   # Literal booleano
sin_valor = None # Literal especial
```

1.2 Variables

Una **variable** es un nombre o identificador que hace referencia a un valor en memoria. En Python, las variables son etiquetas que apuntan a objetos.

- **Declaración e inicialización:** Las variables se crean en el momento de la asignación usando el operador =.

```
x = 10
mensaje = "Hola mundo"
precio = 19.99
```

- **Tipado dinámico:** Python es un lenguaje de tipado dinámico, lo que significa que el tipo de dato de una variable se determina en tiempo de ejecución. Una misma variable puede cambiar de tipo si se le asigna un nuevo valor de tipo diferente.

```
variable = 10    # 'variable' es un entero (int)
```

```
variable = "texto" # Ahora 'variable' es una cadena (str)
```

- **Asignación múltiple y encadenada:** Python permite asignar varios valores a múltiples variables en una sola línea.

```
# Asignación múltiple
```

```
a, b, c = 1, 2, 3
```

```
print(f"a: {a}, b: {b}, c: {c}") # Salida: a: 1, b: 2, c: 3
```

```
# Asignación encadenada
```

```
x = y = z = 100
```

```
print(f"x: {x}, y: {y}, z: {z}") # Salida: x: 100, y: 100, z: 100
```

- **Intercambio de valores:** Una de las características más elegantes de Python es la capacidad de intercambiar los valores de dos variables sin necesidad de una variable temporal.

```
a = 5
```

```
b = 10
```

```
print(f"Antes del intercambio: a={a}, b={b}")
```

```
a, b = b, a
```

```
print(f"Después del intercambio: a={a}, b={b}")
```

1.3 Conversión de tipos

A menudo es necesario convertir un tipo de dato a otro. Para ello, Python proporciona funciones de conversión.

- `int(valor)`: Convierte a entero.
- `float(valor)`: Convierte a flotante.
- `str(valor)`: Convierte a cadena.
- `bool(valor)`: Convierte a booleano (False para 0, None, cadenas o listas vacías; True para el resto).

```
num_str = "150"
```

```
num_int = int(num_str)  # num_int es 150 (entero)
```

```
num_float = float(num_int) # num_float es 150.0 (flotante)
```

```
edad = 25
```

```
edad_str = str(edad)  # edad_str es "25" (cadena)
```

1.4 Identificadores válidos y convenciones

Un **identificador** es el nombre que se le da a una variable, función o clase. Para que un identificador sea válido, debe:

- Comenzar con una letra o un guion bajo (`_`).
- Contener solo letras, números y guiones bajos.
- Distinguir entre mayúsculas y minúsculas (`edad` y `Edad` son variables diferentes).
- No ser una palabra clave reservada del lenguaje (como `if`, `for`, `class`).

Convenciones de nombres (PEP 8):

- **snake_case:** Para variables y funciones. Las palabras se separan con guiones bajos. (ej. nombre_alumno, edad_actual).
- **PascalCase:** Para nombres de clases. Cada palabra empieza con mayúscula. (ej. ClasePersona, MiClase).
- **MAYUSCULAS_CON_GUIONES:** Para constantes. (ej. PI = 3.14, MAX_INTENTOS = 5).

2. Operadores

Un **operador** es un símbolo que le indica al compilador o intérprete que realice una operación matemática, relacional o lógica.

2.1 Operadores aritméticos

Se usan para realizar operaciones matemáticas.

Operador	Significado	Ejemplo	Resultado
+	Suma	5 + 3	8
-	Resta	5 - 3	2
*	Multiplicación	5 * 3	15
/	División real	5 / 2	2.5
//	División entera	5 // 2	2
%	Módulo (resto)	5 % 2	1
**	Potencia	2 ** 3	8

2.2 Operadores relacionales (comparación)

Comparan dos valores y devuelven un valor booleano (True o False).

Operador	Significado	Ejemplo	Resultado
==	Igual a	5 == 5	True
!=	Distinto de	5 != 3	True
<	Menor que	3 < 5	True
>	Mayor que	5 > 3	True
<=	Menor o igual que	3 <= 3	True
>=	Mayor o igual que	5 >= 3	True

2.3 Operadores lógicos

Se usan para combinar o invertir expresiones booleanas.

Operador	Significado	Ejemplo	Resultado
and	Y lógico	True and False	False
or	O lógico	True or False	True
not	Negación lógica	not True	False

2.4 Operadores de asignación compuesta

Permiten asignar un valor a una variable a la vez que se modifica.

Operador Ejemplo Equivalente a

`+=` `x += 2` `x = x + 2`

`-=` `x -= 2` `x = x - 2`

`*=` `x *= 3` `x = x * 3`

`/=` `x /= 2` `x = x / 2`

`//=` `x //= 2` `x = x // 2`

`%=` `x %= 3` `x = x % 3`

`**=` `x **= 2` `x = x ** 2`

2.5 Operadores de pertenencia (in, not in)

Comprueban si un valor se encuentra dentro de una secuencia (lista, cadena, tupla, etc.).

Operador	Significado	Ejemplo	Resultado
in	Está en	'a' in 'python'	False
not in	No está en	'a' not in 'python'	True

2.6 Operadores de identidad (is, is not)

Comparan si dos variables hacen referencia al **mismo objeto en memoria**. Es diferente a ==, que compara solo el valor.

```
a = [1, 2, 3]
```

```
b = a      # 'b' apunta al mismo objeto que 'a'
```

```
c = [1, 2, 3]  # 'c' es un nuevo objeto, aunque su valor es el mismo
```

```
print(a is b)  # True
```

```
print(a is c)  # False
```

3. Expresiones y precedencia

3.1 Concepto de expresión

Una **expresión** es cualquier combinación de literales, variables y operadores que el intérprete de Python puede evaluar para producir un valor.

Ejemplos de expresiones

2 + 3 # Expresión aritmética

a > 5 # Expresión relacional

"Hola " + "mundo" # Expresión con strings

3.2 Expresiones anidadas y uso de paréntesis

Una expresión puede contener otras expresiones. El uso de paréntesis () es recomendable para dejar clara la intención del programador y controlar el orden de evaluación.

Sin paréntesis, se aplica la precedencia

resultado = 2 + 3 * 4 # El resultado es 14 (3*4 se hace primero)

Con paréntesis, se controla la precedencia

resultado_con_parentesis = (2 + 3) * 4 # El resultado es 20 (2+3 se hace primero)

3.3 Precedencia y asociatividad de operadores

La **precedencia** determina el orden en que se evalúan las operaciones. Por ejemplo, la multiplicación (*) tiene mayor precedencia que la suma (+). Cuando varios operadores tienen la misma precedencia, la **asociatividad** determina el orden de evaluación (normalmente de izquierda a derecha).

Precedencia (de mayor a menor) Operadores

1 (Mayor)	() (paréntesis)
2	** (potencia)
3	+x, -x (signos unarios)
4	*, /, //, %
5	+, -
6	==, !=, <, >, <=, >=
7	is, is not, in, not in
8	not
9	and
10 (Menor)	or

3.4 Borrado de variables y ámbito (scope)

El **ámbito (scope)** de una variable define dónde es accesible dentro del programa.

- **Ámbito local:** La variable solo existe dentro de una función.
- **Ámbito global:** La variable existe en todo el programa.

```
x = 10 # Esta es una variable global
```

```
def mi_funcion():
```

```
    y = 5 # Esta es una variable local
```

```
    print("Dentro de la función:", y)
```

```
    print("Accediendo a la global:", x)
```

```
mi_funcion()
```

```
print("Fuera de la función:", x)
```

```
# print(y) # Esto daría un error (NameError) porque 'y' no existe aquí
```

Se puede eliminar una variable usando la instrucción del.

Otro ejemplo:

```
mi_variable = "Ejemplo"
```

```
del mi_variable
```

```
# print(mi_variable) # Esto provocaría un error
```

4. Función eval() (Opcional)

La función eval() es una herramienta poderosa que debe usarse con precaución.

4.1 ¿Qué es eval()?

eval() es una función incorporada que toma una cadena de texto y la ejecuta como una expresión de Python, devolviendo su resultado.

```
expresion_str = "2 * 3 + 1"
resultado = eval(expresion_str)
print(f"El resultado de '{expresion_str}' es: {resultado}")
# Salida: El resultado de '2 * 3 + 1' es: 7
```

4.2 Riesgos de seguridad

El uso de eval() con datos de entrada no confiables es extremadamente peligroso, ya que puede ejecutar cualquier código de Python. Si un atacante inyecta una cadena maliciosa, eval() podría ejecutarla, comprometiendo la seguridad del sistema.

¡¡EJEMPLO PELIGROSO!!

```
cadena_maliciosa = "__import__('os').system('ls -la')"
```

eval(cadena_maliciosa) # Esto podría ejecutar un comando del sistema

No usar eval() con entrada de usuario sin validar.

Por esta razón, su uso está desaconsejado para la mayoría de los casos. Para evaluar expresiones matemáticas de forma segura, se pueden usar módulos como ast.literal_eval() o bibliotecas como sympy.